

Problem 2: Longest Common Substring (case-sensitive)

Alikhan Makhamadaliev

Algorithm (DP):

Let $n = \text{text1.length}$, $m = \text{text2.length}$.

Create a 2D int array of size $(n+1) \times (m+1)$ filled with 0.

$\text{dp}[i][j]$ = length of the longest common substring that ends at $\text{text1}[i-1]$ and $\text{text2}[j-1]$.

$\text{bestLen} = 0$

$\text{bestEnd} = 0$ // end position in text1 (using i)

For i from 1 to n:

 For j from 1 to m:

 If $\text{text1}[i-1] == \text{text2}[j-1]$:

$\text{dp}[i][j] = \text{dp}[i-1][j-1] + 1$

 If $\text{dp}[i][j] > \text{bestLen}$:

$\text{bestLen} = \text{dp}[i][j]$

$\text{bestEnd} = i$

 Else:

$\text{dp}[i][j] = 0$

If $\text{bestLen} == 0$:

 return ""

Else:

 return $\text{text1.substring}(\text{bestEnd} - \text{bestLen}, \text{bestEnd})$

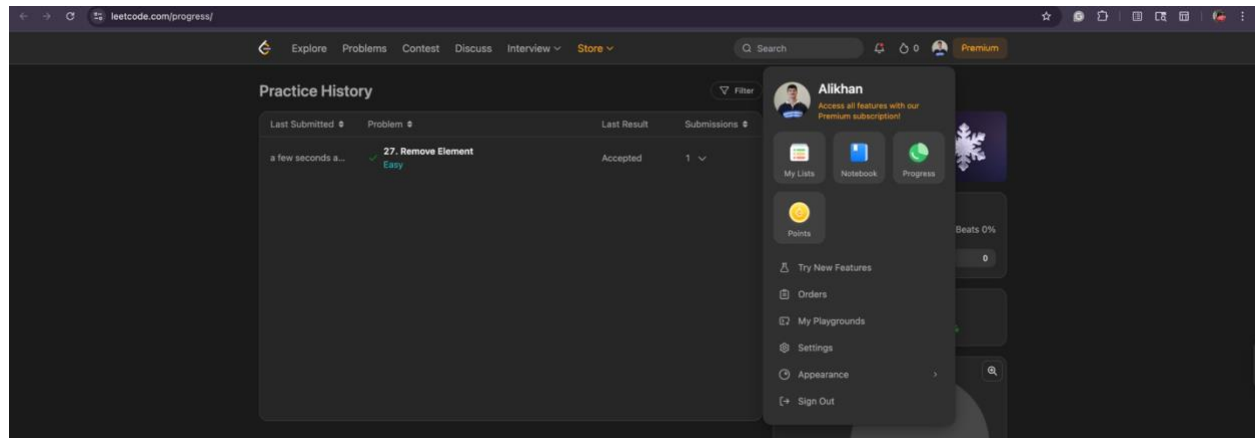
This algorithm works because a common substring must be contiguous, so when characters mismatch, the streak must reset to 0. When characters match, we extend the previous matching streak from $\text{dp}[i-1][j-1]$.

Complexity:

Time: $O(n*m)$ because we fill the dp table once.

Space: $S(n*m)$ for the dp table

Extra Credit:



Problem 6 – Complexity writeup

Problem 1 (LCS length):

Time: $O(n*m)$, $\Omega(n*m)$ because the DP table of size $(n+1)(m+1)$ is filled.

Space: $S(n*m)$ for the dp table.

Problem 2 (Longest Common Substring):

Time: $O(n*m)$, $\Omega(n*m)$ because dp compares all pairs (i, j) .

Space: $S(n*m)$ for dp

Problem 3 (Not Fibonacci first k terms):

Time: $O(k)$ iterations (BigInteger arithmetic scales with the size of the values).

$\Omega(k)$ because you must generate each term to output it.

Space: $S(k)$ if storing all terms in an array, or $S(1)$ if printing as you generate.

Problem 4 (Where in Sequence):

Time: $O(t)$ where t is the number of terms generated until exceeding x .

Best case $\Omega(1)$ if x is 0 or 1.

Space: $S(1)$ because only a few BigIntegers and a counter are kept.

Problem 5 (Remove Elements Smaller Than Val):

Time: $O(n)$, $\Omega(n)$ because every element must be checked once.

Space: $S(1)$ since it overwrites in-place.

Extra Credit BELOW!!

Extra Credit: NotFibonacci Plot (Java)

The NotFibonacci sequence grows extremely fast, so plotting the raw values causes the graph to become unreadable and can overflow fixed-size numeric types. To handle this, the sequence was generated using BigInteger and the plot displays $\log_{10}(\text{values})$ instead of the raw value.

